
Table of Contents

1. Executive Summary	3
Top five risks	3
2. Foundations: lastsaas Architecture Recap	4
Backend stack	4
Frontend stack	4
Existing collections (30+)	5
Key extension points	5
What lastsaas does NOT have (and we must add)	6
3. Target: Tutor LMS Pro Feature Decomposition	7
3.1 Domain model: course authoring	7
3.2 Domain model: quizzes and questions	7
3.3 Domain model: ecommerce and subscriptions	8
3.4 Pro-only feature inventory	9
4. Gap Analysis and Extension Strategy	12
5. Data Model: MongoDB Collection Diff	14
5.1 Course authoring collections	14
5.2 Quiz and assignment collections	14
5.3 Enrollment and progress collections	15
5.4 Ecommerce collections	15
5.5 Pro feature collections	16
6. Backend Module Architecture (Go)	18
6.1 Folder structure	18
6.2 Course module skeleton	18
6.3 Route wiring in main.go	18
7. API Surface Design	20
7.1 Courses and curriculum	20
7.2 Quizzes, assignments, attempts	20
7.3 Enrollment and progress	21
7.4 Ecommerce	22
7.5 Subscriptions and bundles	22

7.6 Certificates, instructors, drip	23
7.7 Reports, gamification, notifications, AI	24
7.8 RBAC matrix (per role)	24
8. Frontend Architecture (React 19 + Vite)	25
8.1 Route map	25
8.2 Feature folders under src/features/lms/	25
8.3 New shared components needed	26
8.4 State management	26
9. Cross-Cutting Concerns	28
9.1 Multi-tenancy isolation	28
9.2 RBAC role extension	28
9.3 Entitlement gating	28
9.4 Event catalog (new event types)	29
10. Video and Media Pipeline	31
10.1 Decision matrix	31
10.2 Upload and playback flow	31
10.3 Content security (Pro)	32
10.4 Object storage abstraction	32
11. Phased 6-Month Roadmap	33
11.1 Phase overview	33
11.2 Phase 0 — Foundation hardening (Week 0, parallel)	33
11.3 Phase 1 — Core LMS (Weeks 1–6)	34
11.4 Phase 2 — Ecommerce (Weeks 7–12)	34
11.5 Phase 3 — Pro authoring (Weeks 13–16)	35
11.6 Phase 4 — Pro engagement (Weeks 17–20)	36
11.7 Phase 5 — Reports + AI + launch (Weeks 21–24)	36
12. Risk Register and Mitigations	38
13. Open Questions and Next Steps	40
13.1 Decisions to confirm before Phase 1 starts	40
13.2 Team and resourcing	40
13.3 Immediate next steps (this week)	40
13.4 What this plan deliberately does NOT include	41

1. Executive Summary

This document is the engineering blueprint for transforming **lastsaas** — a Go 1.25 + React 19 multi-tenant SaaS foundation — into a fully featured Learning Management System that reaches feature parity with **Tutor LMS Pro**, the WordPress plugin published by Themeum. The plan deliberately reuses every reusable primitive that lastsaas already ships (multi-tenancy, JWT auth, RBAC, Stripe billing, entitlement gating, in-process event emitter, webhook delivery, MCP server, audit log) and adds only what the LMS domain genuinely requires: a course authoring surface, a lesson and quiz runtime, an ecommerce flow specialized for digital goods, a certificate engine, a video pipeline, a scheduled content-release runner, and an AI generation layer.

The target product is a multi-tenant B2B SaaS in which each tenant is a *school* — an independent course-selling organization with its own instructors, students, branding, and Stripe account. Within a tenant, multiple instructors can author courses, share revenue, and use a unified student experience. Students enroll in courses either through one-time purchases or recurring subscriptions, progress through drip-released content, take quizzes, submit assignments, and earn auto-issued certificates. The platform operator (you) monetizes via tiered subscription plans gated by feature entitlements, exactly the model Tutor LMS Pro uses for its own pricing.

Delivery is structured into five phases over twenty-four weeks. Phase 0 hardens the foundation (object storage, scheduled jobs, new RBAC roles, new entitlement keys). Phase 1 ships the core LMS (course catalog, curriculum builder, lesson player, progress tracking). Phase 2 adds ecommerce (cart, checkout, coupons, Stripe one-time and recurring, instructor earnings). Phase 3 layers on Pro authoring (assignments, certificates, content drip, multi-instructor revenue share). Phase 4 adds Pro engagement (gamification, Q&A, announcements, lesson notes, notifications, course preview). Phase 5 closes with reports and an AI generation layer, then enters launch hardening. Each phase has explicit exit criteria and ships a demoable slice.

Top five risks

- 1 Video pipeline cost overruns.** Bunny Stream and Mux both bill per minute streamed and per GB egress; an enthusiastic instructor uploading 50 hours of 1080p video without policy guardrails can blow through a hosting budget in days. Mitigation: enforce upload size limits, transcode-preset caps, signed-URL expiry, and per-tenant bandwidth budgets surfaced on the admin dashboard.
- 2 Stripe subscription edge cases.** Proration, mid-cycle plan changes, failed renewals (SCA, expired cards), dunning, refund cascades against instructor revenue share — each of these has a non-trivial state machine. Mitigation: build a thin abstraction over Stripe Billing that explicitly models our domain, treat webhooks as the source of truth (idempotent), and write property tests for refund/revenue math.
- 3 MongoDB schema migration pain.** lastsaas has no migration framework — schema lives in Go struct tags. Adding ~35 new collections without breaking changes will be painful. Mitigation: introduce a lightweight migration runner early (Phase 0) using versioned JSON documents in a `_migrations` collection, plus per-collection schema validators in MongoDB.
- 4 Certificate PDF fidelity.** Tutor LMS Pro ships 12 certificate templates with pixel-perfect PDF rendering. lastsaas uses `gofpdf` which lacks font subsetting and modern CSS layout. Mitigation: render certificates via Playwright headless Chrome from HTML templates — same pipeline as the cover of this document — then store the resulting PDF in object storage with a verification hash.
- 5 Multi-instructor revenue math under refunds.** When a course has three instructors with a 40/35/25 split and the student requests a partial refund mid-course, the revenue reversal must respect the same split, net of platform fee, and produce an auditable ledger entry. Mitigation: model revenue as a double-entry ledger from day one, never as derived counters on the order row.

2. Foundations: lastsaas Architecture Recap

lastsaas is a single-tenant-deployable, multi-tenant-by-design SaaS foundation. The entire product ships as one Docker image: a Go binary serves `/api/*` from `gorilla/mux` and serves the built React SPA from `/app/static`. The only external dependency in production is MongoDB Atlas; everything else (email, payments, file blobs) is an external SaaS API. The default deployment target is Fly.io on port 8080.

Backend stack

Layer	Choice	Notes for LMS extension
Language	Go 1.25	Generics used throughout; new modules follow the same patterns.
HTTP framework	gorilla/mux	Routes wired in <code>cmd/server/main.go</code> lines 376–790. Each resource group gets a subrouter.
Database	MongoDB (official driver)	No ORM. Schema maintained in <code>internal/db/schema.go</code> via Go struct tags. ~30 existing collections.
Auth	JWT HS256 (60m access / 30d refresh)	Refresh tokens rotated on use; <code>revoked_tokens</code> collection for logout. RBAC roles: owner, admin, user.
Billing	Stripe v82	<code>stripe.Service.CreateCheckoutSession</code> supports both one-time and recurring modes. <code>stripe_mappings</code> collection links Stripe IDs to internal entities.
Email	Resend	templates rendered via Go <code>html/template</code> . Email queue is in-process.
Events	<code>events.Emitter</code> (in-process pub/sub)	Synchronous, in-memory. Subscribers register in <code>init()</code> . Used for audit log, webhooks, notifications.
Webhooks	Outbound only	Configurable per-tenant. Retry with exponential backoff. Webhook events catalog is the extension point.
Background jobs	In-process goroutines	No external queue. Cron-like scheduling absent — must add for content drip.
File storage	MongoDB GridFS for small blobs	No S3/R2 abstraction. Must add for video — GridFS cannot handle multi-GB files.
PDF generation	<code>gofpdf</code>	Sufficient for invoices; insufficient for certificates — will use Playwright.
MCP server	<code>mark3labs/mcp-go</code>	Exposes CLI tools as MCP resources. LMS admin operations can be exposed as MCP tools.
Config	<code>configstore</code> (runtime, no redeploy)	Used for feature flags and per-tenant settings.
Audit log	<code>audit_log</code> collection	Append-only, indexed by <code>tenant_id</code> + <code>actor_id</code> + timestamp.

Frontend stack

Layer	Choice	Notes for LMS extension
Build	Vite 7	Fast HMR. React 19. No SSR.
Routing	<code>react-router-dom v7</code>	All routes registered in <code>src/App.tsx</code> . LMS routes will be added as a feature sub-tree.

Layer	Choice	Notes for LMS extension
Server state	TanStack Query v5	Used for all API calls. New LMS hooks follow the same <code>useXxxQuery</code> / <code>useXxxMutation</code> naming.
Client state	Zustand	Sparse usage. Will host quiz session state and video player state.
Forms	react-hook-form v7 + zod v4	Schema-first. Course builder form will reuse this.
UI primitives	8 custom components in <code>src/components/ui/</code>	Not shadcn. Will need to add: Tabs, Dialog, Drawer, Command palette, DragDrop context, Rich text editor.
Styling	Tailwind CSS v4 (@theme directive)	Per-component CSS modules not used. LMS components follow Tailwind conventions.
i18n	None	All strings hardcoded. Must add if international schools are a target — open question.
Theming	Light/dark via <code>branding_config</code>	Per-tenant branding already supported. LMS pages inherit.

Existing collections (30+)

These collections are already defined in `internal/db/schema.go`. The LMS extension will add new collections alongside them, all keyed by `tenant_id` for isolation.

Domain	Collections
Identity	<code>users</code> , <code>refresh_tokens</code> , <code>verification_tokens</code> , <code>oauth_states</code> , <code>revoked_tokens</code> , <code>auth_codes</code> , <code>webauthn_credentials</code> , <code>webauthn_sessions</code> , <code>sso_connections</code>
Tenancy	<code>tenants</code> , <code>tenant_memberships</code> , <code>invitations</code>
Billing	<code>plans</code> , <code>credit_bundles</code> , <code>financial_transactions</code> , <code>stripe_mappings</code>
Platform	<code>config_vars</code> , <code>announcements</code> , <code>custom_pages</code> , <code>messages</code> , <code>usage_events</code> , <code>telemetry_events</code> , <code>event_definitions</code> , <code>system_config</code> , <code>system_logs</code> , <code>audit_log</code> , <code>impersonation_logs</code> , <code>branding_config</code> , <code>branding_assets</code>
Integration	<code>api_keys</code> , <code>webhooks</code> , <code>webhook_deliveries</code> , <code>webhook_events</code>
Operations	<code>leader_locks</code> , <code>counters</code> , <code>system_nodes</code> , <code>system_metrics</code> , <code>daily_metrics</code>

Key extension points

lastsaas was designed for extension. Five primitives will be reused as-is, with no schema changes required:

- 1 **events.Emitter** — subscribe to existing events (`user.created`, `tenant.invited`, `stripe.checkout.completed`) and emit new LMS events (`course.published`, `lesson.completed`, `enrollment.created`, `certificate.issued`).
- 2 **RequireEntitlement middleware** — already used to gate features per plan. Adding LMS entitlements is a config change, not a code change. New keys: `lms.quizzes`, `lms.certificates`, `lms.multi_instructor`, `lms.content_drip`, `lms.gamification`, `lms.reports`, `lms.tutorai`, `lms.zoom`.
- 3 **Webhook event catalog** — extend `WebhookEventType` enum with LMS events. Tenants can subscribe to `enrollment.created` to push into their CRM.
- 4 **stripe.Service.CreateCheckoutSession** — already supports both one-time and recurring modes. Course purchase = one-time; subscription plan = recurring. The line-item shape is the only new code.

-
- 5 **configstore** — runtime per-tenant config without redeploy. Course completion thresholds, video quality caps, AI token budgets — all live here.

What lastsaas does NOT have (and we must add)

- Object storage abstraction (S3 / Cloudflare R2 / Bunny Stream).
- Scheduled jobs runner (for content drip, certificate expiry, dunning emails, AI batch generation).
- Rich text / blocks editor on the frontend (recommend TipTap).
- Video pipeline (transcode, signed playback, DRM-lite via token auth).
- Migration framework (MongoDB schema validators + versioned migrations).
- Search index (Mongo text index is enough at MVP scale; Atlas Search or Meilisearch later).
- Any LMS domain code — confirmed greenfield by repo-wide grep for 'course', 'lesson', 'enrollment', 'instructor', 'video'.

3. Target: Tutor LMS Pro Feature Decomposition

Tutor LMS is a WordPress plugin, so its data model is shaped by WordPress conventions: courses, topics (sections), lessons, quizzes, and questions are all custom post types; richer data lives in custom tables (`tutor_quiz_attempts`, `tutor_enrolled`, `tutor_order`, `tutor_subscription_plans`, `tutor_subscriptions`) and the postmeta key/value table. Pro features are wired in through a classic add-on loader at `tutor-pro/tutor-pro/addons/` with a base helper class. This decomposition matters because it tells us what to consolidate into clean document collections and what to keep as separate modules.

3.1 Domain model: course authoring

Entity	Source	Key fields	Relationships
Course	free: models/CourseModel.php	title, description, status (draft/publish/pending/private/future/trash), price, level, category, tags, benefits, requirements, target_audience, total_enrolled, rating_avg, thumbnail, video, completion_mode (flexible/strict)	has_many Topics → Lessons; has_many Instructors (Pro); has_many Enrollments; has_many Reviews; has_many Certificates (Pro)
Topic (Section)	free: WP post type 'topics'	title, summary, course_id, sort_order	belongs_to Course; has_many Lessons
Lesson	free: models/LessonModel.php	title, content, video_source (html5/youtube/vimeo/embedded/bunny), video_url, attachments, duration, sort_order	belongs_to Topic; has_one Quiz (optional); has_many Lesson_Notes (Pro); has_many Comments
Lesson Video	free: lesson meta	source_type, source_url, duration, poster, thumbnails	embedded in Lesson document
Attachment	free: lesson/course meta	file_url, file_type, file_size, title	belongs_to Lesson or Course
Course Category	free: WP taxonomy 'course-category'	name, slug, parent, description	has_many Courses
Course Tag	free: WP taxonomy 'course-tag'	name, slug	has_many Courses
Review	free: course meta	student_id, rating (1-5), review_text, created_at	belongs_to Course, belongs_to Student
Q&A; Question	free: views/qna/	student_id, course_id, lesson_id, question, answers[]	belongs_to Course/Lesson; has_many Answers

3.2 Domain model: quizzes and questions

Quiz attempts are stored in the `tutor_quiz_attempts` table with per-question answers in `tutor_quiz_attempt_answers`. Question types are the most distinctive feature — Tutor LMS supports 14 types, of which 5 are Pro-only.

Question type	Free / Pro	Storage shape
true_false	Free	question, correct_answer (boolean)

Question type	Free / Pro	Storage shape
single_choice	Free	question, options[], correct_option_id
multiple_choice	Free	question, options[], correct_option_ids[]
open_ended	Free	question, expected_answer (free text, instructor-graded)
fill_in_the_blank	Free	question with {blank} placeholders, answers[]
short_answer	Free	question, expected_answer (≤200 chars)
matching	Free	question, pairs[] = {left, right}
image_matching	Free	question, image_pairs[] = {image, label}
image_answering	Free	question, image, correct_answer
ordering	Free	question, items[] (correct order)
draw_image (Image Marking)	Pro	question, image, marking_areas[]
scale (Range)	Pro	question, min, max, step, correct_range
pin_image (Pin)	Pro	question, image, pin_coordinates
coordinates	Pro	question, image, correct_coordinates
puzzle	Pro	question, puzzle_pieces[]

Quiz entity	Source	Key fields	Notes
Quiz	free: models/Quiz Model.php	title, description, time_limit, attempts_allowed, passing_score, grading_mode (auto/manual), shuffle_questions, show_answer_feedback	Attached to a Lesson or standalone
Quiz Attempt	free: tutor_quiz_attempts	student_id, quiz_id, attempt_status (started/ended/review_required/timeout), result (pass/fail/pending), score, started_at, ended_at	Per-question answers stored separately
Quiz Attempt Answer	free: tutor_quiz_attempt_answers	attempt_id, question_id, answer (JSON), is_correct, marks_awarded	One row per question per attempt
Question Bank	free	questions reusable across quizzes	Pro add-on adds import/export

3.3 Domain model: ecommerce and subscriptions

Entity	Source	Key fields	Notes
Cart	free: models/CartModel.php	tenant_id, student_id, items[] = {course_id, plan_id, price}	Per-session or per-student
Cart Item	free: models/CartItem Model.php	cart_id, course_id or bundle_id or plan_id, list_price, sale_price	Polymorphic
Order	free: models/OrderModel.php	status (incomplete/completed/cancelled/trash/pending), payment_status (paid/failed/unpaid/refunded/partially-refunded/pending), payment_method (manual/free/stripe/paypal), total, tax, discount, currency	1:N to OrderItems

Entity	Source	Key fields	Notes
Order Item	free: models/OrderItemModel.php	order_id, product_type (course/bundle/subscription), product_id, price	Polymorphic
Order Activity	free: models/OrderActivitiesModel.php	order_id, action (created/paid/refunded/cancelled), actor_id, metadata	Append-only audit log per order
Coupon	free: models/CouponModel.php	code, status (active/inactive/expired/scheduled), discount_type (flat/percentage), discount_value, applies_to (all_courses/specific_courses/category/membership), min_purchase, max_uses, per_user_limit, valid_from, valid_to	Rich — 6 applies_to types
Coupon Redemption	free	coupon_id, order_id, student_id, redeemed_at, discount_applied	Audit + uniqueness
Subscription Plan	Pro: subscription/src/Models/PlanModel.php	name, status (active/inactive), payment_type (onetime/recurring), plan_type (course/bundle/category/full_site), price, interval (hour/day/week/month/year), interval_count, trial_days	Recurring only = membership; full_site = membership
Subscription	Pro: subscription/src/Models/SubscriptionModel.php	student_id, plan_id, status (pending/active/expired/hold/cancelled/trash), started_at, current_period_end, history[]	History events: subscribed, renewed, resubscribed, cancelled, put_on_hold
Bundle	Pro	title, courses[], price	Sold as one unit; enrolled student gets all bundle courses
Withdrawal Request	free: models/WithdrawalModel.php	instructor_id, amount, status (pending/approved/rejected/paid), method, requested_at	Per-instructor payout

3.4 Pro-only feature inventory

These features exist in `tutor-pro/tutor-pro/` but NOT in the free repo. They are the paywall surface for the SaaS — each maps to one or more entitlement keys.

Feature	Pro folder	SaaS entitlement key
Certificates (12 PDF templates, auto-issue, verify URL)	addons/tutor-certificates/	lms.certificates
Multi-instructor + revenue share	addons/tutor-multi-instructors/, classes/Instructor_Percentage.php	lms.multi_instructor
Content Drip (schedule by date / prereq / enrollment)	addons/content-drip/	lms.content_drip
Native Subscriptions & Memberships	addons/subscription/	lms.subscriptions (bundled with plan tier)
Zoom meetings & webinars as lessons	addons/tutor-zoom/	lms.zoom

Feature	Pro folder	SaaS entitlement key
Advanced reports (sales, course, student, earnings, statements)	addons/tutor-report/	lms.reports
Gamification (points, badges, leaderboards) via GamiPress integration	addons (via hooks)	lms.gamification
H5P interactive content	addons/h5p/	lms.h5p
BuddyPress social layer	addons/buddypress/	lms.community (deferred)
Course Preview (free preview lessons)	addons/tutor-course-preview/	lms.course_preview
Course Attachments (downloadable resources)	addons/tutor-course-attachments/	lms.attachments
Custom Email templates (30+)	addons/tutor-email-templates/email/	lms.custom_emails
Push notifications (real-time, via Pusher)	addons/tutor-notifications/	lms.notifications
2FA + reCAPTCHA + OTP login	addons/auth/	lms.advanced_auth (always on)
Social login (Google, Facebook)	addons/social-login/	lms.social_login
Calendar (Google Calendar sync)	addons/calendar/	lms.calendar
Quiz import/export	addons/quiz-import-export/	lms.quiz_import_export
WPML multilingual	addons/tutor-wpml/	lms.i18n (open question)
Google Classroom integration	addons/google-classroom/	lms.gclassroom (deferred)
Paid Memberships Pro integration	addons/pmpro/	N/A — we ship our own
Restrict Content Pro integration	addons/restrict-content-pro/	N/A — we ship our own
Weglot translation	addons/tutor-weglot/	N/A — we ship our own
Lesson Notes	classes/LessonNotes.php	lms.lesson_notes
Content Security (video DRM-lite, watermark)	classes/ContentSecurity.php	lms.content_security
Device Management (limit concurrent logins)	classes/DeviceManagement.php	lms.device_management
Course Coming Soon (pre-launch pages)	classes/CourseComingSoon.php	lms.coming_soon

Feature	Pro folder	SaaS entitlement key
Course Duplicator	classes/Course_Duplicator.php	lms.course_duplicate
Email Verification	classes/EmailVerification.php	always on (lastsaas already does this)
Guest Checkout	ecommerce/GuestCheckout/	lms.guest_checkout
Gift Course	templates/dashboard/gift-course.php, gift-course/	lms.gift_course
Instructor application & review	classes/Instructor.php	lms.instructor_apply (always on)
Webinars (live lessons)	classes/Webinar.php	bundled with lms.zoom
Progress Reset (per-student)	classes/ProgressReset.php	lms.progress_reset
TutorAI (AI course / quiz generation)	tutorai/, openai/	lms.tutorai
Invoice generation	ecommerce/Invoice.php	always on
Content Duplicator (within tenant)	classes/ContentDuplicator.php	bundled with lms.course_duplicate
Notification preferences per student	classes/NotificationPreference.php	always on
Visibility Field Control (lesson gating)	classes/VisibilityFieldControl.php	always on

4. Gap Analysis and Extension Strategy

The gap analysis below pairs each LMS capability we need with what lastsaas already provides. The pattern is consistent: identity, billing, tenancy, and audit come for free; everything LMS-specific is greenfield. The strategy is therefore additive — we build new modules under `backend/internal/lms/` and new feature folders under `frontend/src/features/lms/`, and we extend three existing touchpoints: the route table in `cmd/server/main.go`, the event catalog in `events/`, and the entitlement catalog in `middleware/entitlement.go`.

Capability	lastsaas today	LMS requirement	Strategy
User identity	JWT, RBAC (owner/admin/user), OAuth, WebAuthn	Need instructor + student roles	Extend RBAC role enum; add role membership to <code>tenant_memberships</code>
Multi-tenancy	<code>tenants</code> + <code>tenant_memberships</code>	Each tenant = school	Reuse as-is; all LMS collections get <code>tenant_id</code> index
Billing	Stripe one-time + recurring via <code>stripe.Service</code>	Course = one-time; plan = recurring; bundle = one-time	Reuse <code>stripe.Service</code> ; add LMS-specific line items
Entitlements	<code>RequireEntitlement</code> middleware on <code>plan.entitlements</code> map	8+ new LMS entitlement keys	Add keys to default plan seeds; middleware unchanged
File storage	MongoDB GridFS (small blobs only)	Need video upload + playback + thumbnails + attachments	Add S3/R2 abstraction (Phase 0); Bunny Stream for video
Scheduled jobs	None — only goroutines	Content drip, dunning, cert expiry, AI batches	Add river-style job runner backed by MongoDB (Phase 0)
Email	Resend + Go templates	30+ LMS event emails	Add LMS templates; subscribe to <code>events.Emitter</code>
Webhooks	Outbound, configurable per-tenant	Add LMS events to catalog	Extend <code>WebhookEventType</code> enum
Rich text editor	None on frontend	Course description, lesson content, certificate text	Add TipTap (Phase 1); store as HTML; sanitize on render
Search	None (manual MongoDB queries)	Course catalog search, lesson search	Phase 1: MongoDB text index. Phase 5: Atlas Search if needed
PDF generation	<code>gofpdf</code> (invoices only)	Certificates with pixel-perfect templates	Render via Playwright from HTML templates; cache to object storage
Video pipeline	None	Upload, transcode, signed playback, watermark	Bunny Stream (preferred) or Mux — Phase 0 decision
Notifications	In-app messages + announcements	Push + email + in-app, per-student preferences	Build on <code>events.Emitter</code> + Pusher for realtime push

Capability	lastsaas today	LMS requirement	Strategy
Analytics / reports	usage_events + telemetry_events (platform-only)	Per-tenant sales, enrollment, completion, earnings	New domain module; aggregations on MongoDB (defer ClickHouse)
AI generation	None	TutorAI: generate course outline, lessons, quizzes	LLM skill (z-ai-web-dev-sdk or OpenAI); prompt engineering layer
i18n	None	Multi-language schools	Open question — defer to post-launch
Migration framework	None	Must not break existing collections	Add lightweight migration runner in Phase 0

5. Data Model: MongoDB Collection Diff

All new collections are document-shaped to exploit MongoDB's strengths. Every collection carries `tenant_id` as the first field after `_id` and is indexed on it. The naming convention is singular nouns (course not courses) to match lastsaas's existing style. Sortable lists use `sort_order` as a float so insertions between two siblings are $O(1)$.

5.1 Course authoring collections

Collection	Key fields	Indexes
course	<code>_id</code> , <code>tenant_id</code> , <code>title</code> , <code>slug</code> , <code>description</code> (HTML), <code>status</code> , <code>level</code> (beginner/intermediate/advanced), <code>price</code> , <code>compare_at_price</code> , <code>currency</code> , <code>category_id</code> , <code>tags[]</code> , <code>thumbnail_url</code> , <code>promo_video_url</code> , <code>benefits[]</code> , <code>requirements[]</code> , <code>target_audience[]</code> , <code>total_enrolled_count</code> , <code>rating_avg</code> , <code>rating_count</code> , <code>completion_mode</code> , <code>created_by</code> , <code>created_at</code> , <code>updated_at</code> , <code>published_at</code>	<code>tenant_id+status+published_at</code> ; <code>tenant_id+slug</code> (unique); <code>tenant_id+category_id</code> ; <code>tenant_id+tags</code> ; <code>text(title,description)</code>
course_topic	<code>_id</code> , <code>tenant_id</code> , <code>course_id</code> , <code>title</code> , <code>summary</code> , <code>sort_order</code>	<code>tenant_id+course_id+sort_order</code>
lesson	<code>_id</code> , <code>tenant_id</code> , <code>course_id</code> , <code>topic_id</code> , <code>title</code> , <code>content</code> (HTML), <code>sort_order</code> , <code>lesson_type</code> (video/text/quiz/zoom/assignment), <code>video</code> { <code>source_type</code> , <code>source_url</code> , <code>duration</code> , <code>poster</code> , <code>thumbnails[]</code> }, <code>attachments[]</code> , <code>is_preview</code> , <code>is_drip_gated</code> , <code>drip_rule_id</code> , <code>created_at</code>	<code>tenant_id+course_id+sort_order</code> ; <code>tenant_id+topic_id</code>
course_category	<code>_id</code> , <code>tenant_id</code> , <code>name</code> , <code>slug</code> , <code>parent_id</code> , <code>description</code> , <code>sort_order</code> , <code>course_count</code>	<code>tenant_id+slug</code> (unique); <code>tenant_id+parent_id</code>
course_tag	<code>_id</code> , <code>tenant_id</code> , <code>name</code> , <code>slug</code> , <code>course_count</code>	<code>tenant_id+slug</code> (unique)
course_review	<code>_id</code> , <code>tenant_id</code> , <code>course_id</code> , <code>student_id</code> , <code>rating</code> (1-5), <code>review_text</code> , <code>instructor_reply</code> { <code>text</code> , <code>created_at</code> }, <code>created_at</code>	<code>tenant_id+course_id</code> ; <code>tenant_id+student_id+course_id</code> (unique)

5.2 Quiz and assignment collections

Collection	Key fields	Indexes
quiz	<code>_id</code> , <code>tenant_id</code> , <code>lesson_id</code> (nullable), <code>title</code> , <code>description</code> , <code>time_limit_seconds</code> , <code>attempts_allowed</code> , <code>passing_score_percent</code> , <code>grading_mode</code> (auto/manual), <code>shuffle_questions</code> , <code>show_answer_feedback</code> , <code>questions[]</code> (embedded)	<code>tenant_id+lesson_id</code>
quiz_question	<code>_id</code> , <code>tenant_id</code> , <code>quiz_id</code> , <code>question_type</code> , <code>question_text</code> , <code>marks</code> , <code>options[]</code> / <code>pairs[]</code> / <code>image_url</code> / <code>scale</code> { <code>min</code> , <code>max</code> , <code>step</code> } / <code>puzzle_pieces[]</code> / <code>correct_answer</code> , <code>explanation</code> , <code>sort_order</code>	<code>tenant_id+quiz_id+sort_order</code>
quiz_attempt	<code>_id</code> , <code>tenant_id</code> , <code>quiz_id</code> , <code>student_id</code> , <code>status</code> (started/ended/review_required/timeout), <code>result</code> (pass/fail/pending), <code>score_percent</code> , <code>started_at</code> , <code>ended_at</code> , <code>time_spent_seconds</code>	<code>tenant_id+student_id+quiz_id</code> ; <code>tenant_id+quiz_id+status</code>
quiz_attempt_answer	<code>_id</code> , <code>tenant_id</code> , <code>attempt_id</code> , <code>question_id</code> , <code>answer</code> (mixed JSON), <code>is_correct</code> , <code>marks_awarded</code> , <code>instructor_feedback</code>	<code>tenant_id+attempt_id</code> ; <code>tenant_id+attempt_id+question_id</code> (unique)

Collection	Key fields	Indexes
assignment	_id, tenant_id, lesson_id, title, brief (HTML), time_limit_seconds, total_marks, pass_marks, allow_file_upload, allowed_file_types[], max_file_size_mb	tenant_id+lesson_id
assignment_submission	_id, tenant_id, assignment_id, student_id, file_urls[], text_answer, submitted_at, eval{status (pending/evaluated), awarded_marks, instructor_feedback, evaluated_at, evaluated_by}	tenant_id+assignment_id+student_id

5.3 Enrollment and progress collections

Collection	Key fields	Indexes
enrollment	_id, tenant_id, course_id, student_id, status (active/cancelled/expired/completed), source (free/purchase/subscription/bundle/admin/gift), order_id, subscription_id, enrolled_at, expires_at, completed_at, progress_percent	tenant_id+student_id+course_id (unique); tenant_id+course_id+status
lesson_progress	_id, tenant_id, enrollment_id, lesson_id, status (not_started/in_progress/completed), time_spent_seconds, last_position_seconds, first_viewed_at, completed_at	tenant_id+enrollment_id+lesson_id (unique); tenant_id+enrollment_id
quiz_attempt	(see 5.2)	
wishlist	_id, tenant_id, student_id, course_id, added_at	tenant_id+student_id+course_id (unique)
lesson_note	_id, tenant_id, student_id, lesson_id, note_text, created_at, updated_at	tenant_id+student_id+lesson_id
qna_question	_id, tenant_id, course_id, lesson_id (nullable), student_id, question, upvotes[], status (open/answered/closed), created_at	tenant_id+course_id+status; tenant_id+lesson_id
qna_answer	_id, tenant_id, question_id, author_id, author_role (student/instructor), answer, is_instructor_answer, created_at	tenant_id+question_id
announcement	_id, tenant_id, course_id, author_id, title, body (HTML), sent_at, read_by[]	tenant_id+course_id+sent_at

5.4 Ecommerce collections

Collection	Key fields	Indexes
cart	_id, tenant_id, owner_id (student or session), items[] = {item_type, item_id, plan_id, list_price, sale_price}, coupon_code, created_at, updated_at	tenant_id+owner_id (unique)
order	_id, tenant_id, student_id (nullable for guest), status, payment_status, payment_method, currency, subtotal, discount, tax, total, billing{email, name, country, address}, created_at, paid_at, refunded_at	tenant_id+status+created_at; tenant_id+student_id+created_at
order_item	_id, tenant_id, order_id, product_type (course/bundle/subscription_plan), product_id, plan_id (nullable), price, discount, tax	tenant_id+order_id

Collection	Key fields	Indexes
order_activity	_id, tenant_id, order_id, action (created/paid/refunded/partially_refunded/cancelled/failed), actor_id, metadata, created_at	tenant_id+order_id+created_at
coupon	_id, tenant_id, code, status, discount_type (flat/percentage), discount_value, applies_to_type, applies_to_ids[], min_purchase_amount, max_uses, per_user_limit, used_count, valid_from, valid_to	tenant_id+code (unique); tenant_id+status
coupon_redemption	_id, tenant_id, coupon_id, order_id, student_id, discount_applied, redeemed_at	tenant_id+coupon_id+order_id (unique); tenant_id+student_id+coupon_id
subscription_plan	_id, tenant_id, name, status, payment_type (onetime/recurring), plan_type (course/bundle/category/full_site), target_id (course_id/bundle_id/category_id/null for full_site), price, currency, interval (hour/day/week/month/year), interval_count, trial_days, sort_order	tenant_id+status; tenant_id+plan_type+target_id
subscription	_id, tenant_id, student_id, plan_id, status (pending/active/expired/hold/cancelled), started_at, current_period_start, current_period_end, cancelled_at, history[] = {event, at, metadata}	tenant_id+student_id+status; tenant_id+plan_id
bundle	_id, tenant_id, title, slug, description, courses[] (with weights for revenue share), price, currency, thumbnail_url	tenant_id+slug (unique)
withdrawal_request	_id, tenant_id, instructor_id, amount, currency, status (pending/approved/rejected/paid), method, payout_reference, requested_at, processed_at	tenant_id+instructor_id+status
revenue_ledger	_id, tenant_id, order_id, instructor_id, role (primary/co_instructor/platform), gross_amount, platform_fee, instructor_share, currency, created_at	tenant_id+instructor_id+created_at; tenant_id+order_id

5.5 Pro feature collections

Collection	Key fields	Indexes
certificate_template	_id, tenant_id, name, html (Jinja-style with placeholders), css, thumbnail_url, orientation (landscape/portrait), is_default	tenant_id
certificate	_id, tenant_id, student_id, course_id, template_id, certificate_number (unique), issued_at, pdf_url, verification_hash, instructor_signatures[] = {instructor_id, signature_image_url}	tenant_id+student_id+course_id (unique); tenant_id+certificate_number (unique); verification_hash (unique)
instructor	_id, tenant_id, user_id, status (pending/approved/rejected), bio, expertise[], social_links{}, payout_method, payout_details{}, commission_rate_default, created_at, approved_at	tenant_id+user_id (unique); tenant_id+status
course_instructor	_id, tenant_id, course_id, instructor_id, role (primary/co), revenue_share_percent, sort_order	tenant_id+course_id; tenant_id+instructor_id (unique)

Collection	Key fields	Indexes
drip_rule	_id, tenant_id, course_id (nullable for lesson-level), lesson_id (nullable), drip_type (schedule/prerequisite/enrollment_days), schedule_at, prerequisite_lesson_id, days_after_enrollment	tenant_id+course_id; tenant_id+lesson_id (unique)
gamification_point	_id, tenant_id, student_id, course_id (nullable), points, reason, awarded_at	tenant_id+student_id; tenant_id+student_id+course_id
gamification_badge	_id, tenant_id, name, slug, description, icon_url, criteria{type, threshold}, is_auto_award	tenant_id+slug (unique)
gamification_student_badge	_id, tenant_id, student_id, badge_id, awarded_at	tenant_id+student_id+badge_id (unique)
gamification_leaderboard	_id, tenant_id, scope (course/tenant), scope_id, period (all_time/monthly), entries[] = {student_id, points, rank}	tenant_id+scope+scope_id+period (unique); rebuilt weekly
notification	_id, tenant_id, user_id, type, title, body, payload{}, read_at, created_at	tenant_id+user_id+created_at; tenant_id+user_id+read_at
notification_preference	_id, tenant_id, user_id, channels = {email, push, in_app}, event_subscriptions{}	tenant_id+user_id (unique)
device_session	_id, tenant_id, user_id, device_fingerprint, user_agent, ip, last_seen_at, is_revoked	tenant_id+user_id; tenant_id+user_id+device_fingerprint (unique)
ai_generation_job	_id, tenant_id, user_id, prompt, target_type (course_outline/lesson_content/quiz/questions), target_id, status (pending/running/succeeded/failed), result{}, tokens_used, llm_model, created_at	tenant_id+status+created_at; tenant_id+user_id
coming_soon_page	_id, tenant_id, course_id, email_capture[], launch_at, description	tenant_id+course_id (unique)

6. Backend Module Architecture (Go)

Each LMS module lives under `backend/internal/lms/<module>/` and follows lastsaas's existing five-file pattern: `handlers.go` (HTTP), `service.go` (business logic), `repository.go` (MongoDB access), `models.go` (struct definitions + BSON tags), `hooks.go` (event subscriptions). Routes are registered in `cmd/server/main.go` under a new `/api/v1/lms` subrouter that gets the standard tenant+auth middleware applied. The example below shows the skeleton for the course module; every other module follows the same shape.

6.1 Folder structure

```
backend/internal/lms/ |— course/ # Course CRUD, publish, curriculum | |— handlers.go
| |— service.go | |— repository.go | |— models.go | |— hooks.go # subscribes to
course.published → emits course.enrollment_opened |— lesson/ # Lessons, topics,
attachments |— quiz/ # Quizzes, questions, attempts, grading |— assignment/ #
Assignments, submissions, grading |— enrollment/ # Enrollments, progress tracking |—
ecommerce/ # Cart, checkout, orders, coupons, invoices |— subscription/ # Plans,
subscriptions, dunning |— certificate/ # Templates, issuance, verification |—
instructor/ # Applications, revenue share, payouts |— drip/ # Drip rules, scheduler
|— gamification/ # Points, badges, leaderboards |— report/ # Aggregated analytics |—
notification/ # In-app, email, push |— media/ # Video upload, signed playback,
thumbnails |— ai/ # TutorAI generation jobs |— shared/ # Shared types, enums, error
codes
```

6.2 Course module skeleton

```
// backend/internal/lms/course/models.go package course type Course struct { ID
primitive.ObjectID `bson:"_id"` TenantID primitive.ObjectID `bson:"tenant_id"` Title
string `bson:"title"` Slug string `bson:"slug"` Description string
`bson:"description"` Status string `bson:"status"` //
draft|pending|publish|private|future|trash Level string `bson:"level"` Price
money.Money `bson:"price"` CategoryID *primitive.ObjectID `bson:"category_id"` Tags
[]string `bson:"tags"` ThumbnailURL string `bson:"thumbnail_url"` Benefits []string
`bson:"benefits"` Requirements []string `bson:"requirements"` CompletionMode string
`bson:"completion_mode"` // flexible|strict CreatedBy primitive.ObjectID
`bson:"created_by"` PublishedAt *time.Time `bson:"published_at"` CreatedAt time.Time
`bson:"created_at"` UpdatedAt time.Time `bson:"updated_at"` }

// backend/internal/lms/course/handlers.go (excerpt) func (h *Handler) Register(r
*mux.Router) { r.HandleFunc("/courses", h.List).Methods("GET")
r.HandleFunc("/courses", h.Create).Methods("POST") r.HandleFunc("/courses/{id}",
h.Get).Methods("GET") r.HandleFunc("/courses/{id}", h.Update).Methods("PATCH")
r.HandleFunc("/courses/{id}/publish", h.Publish).Methods("POST")
r.HandleFunc("/courses/{id}/curriculum", h.GetCurriculum).Methods("GET")
r.HandleFunc("/courses/{id}/curriculum", h.SaveCurriculum).Methods("PUT") } func (h
*Handler) Publish(w http.ResponseWriter, r *http.Request) { ctx := r.Context() courseID
:= chiutil.PathID(r, "id") course, err := h.svc.Publish(ctx, courseID) if err != nil {
writeErr(w, err); return } h.events.Emit(ctx, "course.published", map[string]any{
"course_id": courseID, "tenant_id": course.TenantID, }) writeJSON(w, course) }
```

6.3 Route wiring in main.go

All LMS routes are mounted under `/api/v1/lms`. The subrouter inherits tenant extraction, JWT auth, and audit logging from the parent. Per-route entitlement gating is applied via

middleware.RequireEntitlement.

```
// cmd/server/main.go (excerpt, added near line ~600) lmsRouter :=
apiRouter.PathPrefix("/lms").Subrouter() lmsRouter.Use(middleware.RequireAuth)
lmsRouter.Use(middleware.RequireTenant) // Public student-facing (entitlement-free)
course.NewHandler(svc, events).Register(lmsRouter) // GET /courses is public within
tenant // Instructor-only instructorOnly :=
lmsRouter.PathPrefix("/instructor").Subrouter()
instructorOnly.Use(middleware.RequireRole("instructor", "admin", "owner"))
course.NewInstructorHandler(svc).Register(instructorOnly) // Admin-only adminOnly :=
lmsRouter.PathPrefix("/admin").Subrouter()
adminOnly.Use(middleware.RequireRole("admin", "owner"))
report.NewHandler(svc).Register(adminOnly) // Entitlement-gated certRouter :=
lmsRouter.PathPrefix("/certificates").Subrouter()
certRouter.Use(middleware.RequireEntitlement(db, "lms.certificates"))
certificate.NewHandler(svc).Register(certRouter)
```

7. API Surface Design

The LMS REST surface adds roughly 85 endpoints to lastsaas's existing API. All paths are prefixed with `/api/v1/lms`. The tables below group endpoints by resource and annotate each with its access tier (P = public within tenant, S = student, I = instructor, A = admin/owner, W = webhook).

7.1 Courses and curriculum

Method	Path	Purpose	Access
GET	/courses	List published courses (filter by category, tag, level, price, search, sort)	P
GET	/courses/{slug}	Get course detail by slug (curriculum tree, instructors, reviews, price, plans)	P
POST	/instructor/courses	Create course (draft)	I
GET	/instructor/courses	List instructor's own courses (any status)	I
PATCH	/instructor/courses/{id}	Update course fields	I
POST	/instructor/courses/{id}/publish	Publish (or schedule)	I
POST	/instructor/courses/{id}/duplicate	Clone course within tenant	I
DELETE	/instructor/courses/{id}	Soft delete (status=trash)	I
GET	/instructor/courses/{id}/curriculum	Get topics + lessons + quizzes as nested tree	I
PUT	/instructor/courses/{id}/curriculum	Bulk save curriculum tree (reorder, add, delete)	I
POST	/instructor/courses/{id}/topics	Add topic	I
PATCH	/instructor/courses/{id}/topics/{tid}	Update topic	I
POST	/instructor/courses/{id}/topics/{tid}/lessons	Add lesson to topic	I
PATCH	/instructor/lessons/{lid}	Update lesson	I
POST	/instructor/lessons/{lid}/video/upload-url	Get presigned upload URL (Bunny)	I
POST	/instructor/lessons/{lid}/video/confirm	Confirm upload, trigger transcode	I

7.2 Quizzes, assignments, attempts

Method	Path	Purpose	Access
POST	/instructor/quizzes	Create quiz	I
PATCH	/instructor/quizzes/{id}	Update quiz (settings + questions[] embedded)	I
POST	/instructor/quizzes/{id}/questions	Add question	I
POST	/instructor/quizzes/{id}/import	Import questions JSON (Pro)	I
GET	/instructor/quizzes/{id}/attempts	List attempts for this quiz	I
POST	/instructor/quiz-attempts/{id}/review	Submit instructor review for open-ended questions	I
POST	/student/quizzes/{id}/attempt	Start new attempt	S
GET	/student/quiz-attempts/{id}	Get attempt with questions	S
POST	/student/quiz-attempts/{id}/answer	Submit answer to one question	S
POST	/student/quiz-attempts/{id}/submit	End attempt, compute score	S
POST	/instructor/assignments	Create assignment	I
POST	/student/assignments/{id}/submit	Submit assignment (files + text)	S
POST	/instructor/assignment-submissions/{id}/evaluate	Grade + feedback	I

7.3 Enrollment and progress

Method	Path	Purpose	Access
GET	/student/enrollments	List my enrollments	S
POST	/student/courses/{id}/enroll	Enroll in free course or after-checkout	S
GET	/student/courses/{id}/progress	Get my progress (lesson completion map)	S
POST	/student/lessons/{id}/complete	Mark lesson complete (server validates)	S
POST	/student/lessons/{id}/progress	Heartbeat video position	S
POST	/instructor/courses/{id}/enrollments	Bulk enroll students	I
DELETE	/instructor/enrollments/{id}	Cancel enrollment (admin action)	I
POST	/instructor/enrollments/{id}/reset-progress	Reset student progress	I

Met hod	Path	Purpose	Acc ess
GET	/student/wishlist	List my wishlist	S
POST	/student/wishlist/{courseId}	Add to wishlist	S
DELETE	/student/wishlist/{courseId}	Remove from wishlist	S
POST	/student/lessons/{id}/notes	Save lesson note	S
GET	/student/lessons/{id}/notes	Get my notes for lesson	S

7.4 Ecommerce

Met hod	Path	Purpose	Acc ess
GET	/cart	Get my cart	S/P
POST	/cart/items	Add course or bundle or plan to cart	S/P
PATCH	/cart/items/{id}	Update quantity / remove	S
POST	/cart/apply-coupon	Apply coupon code	S
DELETE	/cart/coupon	Remove coupon	S
POST	/checkout	Create Stripe Checkout session	S/P
POST	/webhooks/stripe	Stripe webhook (idempotent)	W
GET	/student/orders	List my orders	S
GET	/student/orders/{id}/invoice	Download invoice PDF	S
POST	/instructor/coupons	Create coupon	I
GET	/instructor/coupons	List coupons	I
GET	/instructor/orders	List orders for instructor's courses	I
POST	/instructor/withdrawals	Request withdrawal	I
GET	/admin/orders	All orders in tenant	A
POST	/admin/orders/{id}/refund	Issue refund (partial or full)	A
GET	/admin/revenue-ledger	Revenue ledger (filter by instructor, course, date)	A

7.5 Subscriptions and bundles

Method	Path	Purpose	Access
POST	/instructor/subscription-plans	Create plan (course/bundle/category/full_site)	I
GET	/instructor/subscription-plans	List plans	I
PATCH	/instructor/subscription-plans/{id}	Update plan	I
GET	/student/subscriptions	List my active + history	S
POST	/student/subscriptions/{id}/cancel	Cancel (effective at period end or immediately)	S
POST	/student/subscriptions/{id}/resume	Resume a cancelled-but-not-expired sub	S
POST	/instructor/bundles	Create bundle	I
PATCH	/instructor/bundles/{id}	Update bundle	I
GET	/admin/subscriptions	All subscriptions in tenant	A

7.6 Certificates, instructors, drip

Method	Path	Purpose	Access
GET	/instructor/certificate-templates	List templates	I
POST	/instructor/certificate-templates	Create template (HTML+CSS)	I
POST	/instructor/courses/{id}/certificate/issue	Manually issue (auto-issue happens on completion)	I
GET	/student/certificates	List my certificates	S
GET	/student/certificates/{id}/download	Download PDF	S
GET	/certificates/verify/{hash}	Public verification page	P
POST	/instructor/apply	Student applies to become instructor	S
GET	/admin/instructor-applications	List pending applications	A
POST	/admin/instructor-applications/{id}/approve	Approve	A
POST	/instructor/courses/{id}/instructors	Add co-instructor with revenue_share_percent	I
PATCH	/instructor/course-instructors/{id}	Update revenue share	I
POST	/instructor/courses/{id}/drip-rules	Create drip rule for lesson	I
GET	/student/lessons/{id}/drip-status	Get unlock status + countdown	S

7.7 Reports, gamification, notifications, AI

Method	Path	Purpose	Access
GET	/admin/reports/overview	KPIs: revenue, enrollments, active students, completion rate	A
GET	/admin/reports/sales	Sales by day/week/month, by course, by coupon	A
GET	/admin/reports/enrollments	Enrollment trends	A
GET	/admin/reports/completion	Completion funnel per course	A
GET	/instructor/reports/earnings	Instructor's own earnings, by period	I
GET	/instructor/reports/students	Student progress for instructor's courses	I
GET	/student/leaderboard/{scope}	Get leaderboard (course or tenant, period)	S
GET	/student/badges	List my badges	S
GET	/notifications	List my notifications	S
PATCH	/notifications/{id}/read	Mark read	S
POST	/student/notification-preferences	Set per-event channel preferences	S
POST	/instructor/ai/generate-course-outline	TutorAI: generate course outline from prompt	I
POST	/instructor/ai/generate-lesson	Generate lesson content from outline	I
POST	/instructor/ai/generate-quiz	Generate quiz from lesson	I
GET	/instructor/ai/jobs/{id}	Poll job status	I

7.8 RBAC matrix (per role)

Capability	Student	Instructor	Admin	Owner
Browse course catalog	✓	✓	✓	✓
Enroll / purchase	✓	✓	✓	✓
Author course	—	✓ (own)	✓	✓
Approve instructors	—	—	✓	✓
Manage tenant billing	—	—	—	✓
Issue refunds	—	—	✓	✓
View all tenant reports	—	—	✓	✓
View own earnings only	—	✓	✓	✓
Configure entitlements per plan	—	—	—	✓ (via platform super-admin)

8. Frontend Architecture (React 19 + Vite)

The frontend reuses lastsaas's existing shell — the same auth flow, the same TanStack Query client, the same Tailwind theme, the same `components/ui/` primitives. Three new experience shells are introduced under `src/pages/`: a **Public** shell for catalog browsing and checkout (unauthenticated-aware), a **Student Dashboard** shell, and an **Instructor Dashboard** shell. Admin pages live alongside lastsaas's existing admin section.

8.1 Route map

```
frontend/src/pages/ |— public/ # No auth required | |— CoursesPage.tsx # /courses |
|— CourseDetailPage.tsx # /courses/:slug | |— CheckoutPage.tsx # /checkout | |—
|— CartPage.tsx # /cart | |— InstructorProfilePage.tsx | |— CertificateVerifyPage.tsx #
|— /certificates/verify/:hash |— student/ # Auth required, role=student | |—
|— DashboardPage.tsx # /my | |— MyCoursesPage.tsx # /my/courses | |— LearningPage.tsx #
|— /learn/:courseId/:lessonId (course player) | |— MyOrdersPage.tsx # /my/orders | |—
|— MyCertificatesPage.tsx | |— MySubscriptionsPage.tsx | |— WishlistPage.tsx | |—
|— MyQAPage.tsx | |— NotificationSettingsPage.tsx |— instructor/ # Auth required,
|— role=instructor | |— DashboardPage.tsx # /instructor | |— MyCoursesPage.tsx #
|— /instructor/courses | |— CourseBuilderPage.tsx # /instructor/courses/:id/builder |
| |— StudentsPage.tsx # /instructor/students | |— EarningsPage.tsx #
|— /instructor/earnings | |— WithdrawalsPage.tsx | |— QuizzesPage.tsx | |—
|— AssignmentsPage.tsx | |— AiStudioPage.tsx # /instructor/ai |— admin/ # Auth required,
|— role=admin/owner |— CoursesPage.tsx |— InstructorsPage.tsx |— StudentsPage.tsx |—
|— OrdersPage.tsx |— CouponsPage.tsx |— ReportsPage.tsx |— PlansPage.tsx # Subscription
|— plan management
```

8.2 Feature folders under `src/features/lms/`

Feature folder	Hooks	Components
course	<code>useCourseQuery</code> , <code>useCoursesQuery</code> , <code>useCreateCourseMutation</code> , <code>usePublishCourseMutation</code>	<code>CourseCard</code> , <code>CourseGrid</code> , <code>CourseFilterBar</code> , <code>CourseDetailHeader</code> , <code>CurriculumTree</code>
lesson	<code>useLessonQuery</code> , <code>useCompleteLessonMutation</code> , <code>useLessonProgressMutation</code>	<code>LessonSidebar</code> , <code>LessonContent</code> , <code>LessonAttachments</code> , <code>LessonComments</code>
quiz	<code>useQuizQuery</code> , <code>useStartAttemptMutation</code> , <code>useSubmitAnswerMutation</code> , <code>useEndAttemptMutation</code>	<code>QuizRenderer</code> (per-question-type renderer registry), <code>QuestionEditor</code> , <code>AttemptSummary</code> , <code>QuizTimer</code>
assignment	<code>useAssignmentQuery</code> , <code>useSubmitAssignmentMutation</code>	<code>AssignmentUploader</code> , <code>AssignmentGradeCard</code>
enrollment	<code>useEnrollmentsQuery</code> , <code>useCourseProgressQuery</code>	<code>ProgressRing</code> , <code>ContinueLearningCard</code>
ecommerce	<code>useCartQuery</code> , <code>useAddToCartMutation</code> , <code>useCheckoutMutation</code> , <code>useCouponMutation</code>	<code>CartDrawer</code> , <code>CheckoutForm</code> , <code>PriceTag</code> , <code>CouponInput</code> , <code>OrderReceipt</code>
subscription	<code>useSubscriptionsQuery</code> , <code>useCancelSubscriptionMutation</code>	<code>PlanCard</code> , <code>SubscriptionStatus</code> , <code>PlanPicker</code>

Feature folder	Hooks	Components
certificate	useCertificatesQuery, useDownloadCertificateMutation	CertificatePreview, CertificateVerify
instructor	useInstructorCoursesQuery, useEarningsQuery	RevenueShareEditor, InstructorPicker, EarningsChart
drip	useDripStatusQuery	DripBadge, DripRuleEditor
gamification	useLeaderboardQuery, useMyBadgesQuery	LeaderboardTable, BadgeWall, PointsDisplay
notification	useNotificationsQuery, useMarkReadMutation	NotificationBell, NotificationList, PreferenceMatrix
media	useUploadUrlMutation, useConfirmUploadMutation	VideoPlayer (source adapters: bunny, mux, youtube, vimeo, embed), VideoUploader, ThumbnailPicker
ai	useGenerateCourseMutation, useGenerateLessonMutation, useGenerateQuizMutation, useAiJobPoll	AiStudioPanel, OutlineEditor, JobProgress
builder	— (composes others)	CurriculumBuilder (drag-drop tree), LessonTypeSwitcher, QuestionBankPicker

8.3 New shared components needed

Beyond the 8 existing UI primitives in `src/components/ui/`, the LMS surface needs:

- **Tabs** — for course-detail sections (Overview / Curriculum / Instructors / Reviews / Q&A) and dashboard sidebar tabs.
- **Dialog** (modal) — for the certificate preview, coupon picker, refund dialog, AI generation preview.
- **Drawer** — for cart preview, filter panel, mobile navigation.
- **DragDropContext** wrapper (recommend @dnd-kit) — for curriculum builder, quiz question reordering, lesson reordering.
- **RichTextEditor** (recommend TipTap) — for course description, lesson content, certificate text, announcements. Output sanitized HTML.
- **CommandPalette** — for instructor / admin quick navigation.
- **DataGrid** — for admin tables (orders, students, enrollments) with sort/filter/pagination/column-visibility.
- **Chart primitives** — for reports. Recommend Recharts (already React 19 compatible). Line/Bar/Area/Pie + a KPI card component.

8.4 State management

TanStack Query remains the source of truth for all server state. New query keys are namespaced under `['lms', <domain>, ...]` for predictable invalidation. Two Zustand stores are added:

- **useQuizSessionStore** — holds the in-progress attempt: current question index, drafted answers, time remaining, paused state. Synced to `localStorage` for resilience against refresh; cleared on submit.

-
- **usePlayerStore** — holds the current video position, playback rate, fullscreen state, watermark overlay position. Not persisted.

9. Cross-Cutting Concerns

9.1 Multi-tenancy isolation

Every new LMS collection carries `tenant_id` as the first field after `_id` and is indexed on a compound of `tenant_id` plus the primary query field. The existing `middleware.RequireTenant` middleware extracts `tenant_id` from the JWT or API key and injects it into the request context. Every repository method accepts the context and filters by `tenant_id`. A code audit pass in Phase 0 verifies no LMS query bypasses the tenant filter.

9.2 RBAC role extension

lastsaas ships three roles: `owner`, `admin`, `user`. We extend with two LMS-specific roles that are scoped to a tenant membership:

Role	Holds	Can do
instructor	Any tenant member who passed instructor application review	All student capabilities + author own courses + view own earnings + manage own co-instructors
student	Any tenant member (default role at signup)	Browse catalog, enroll, take lessons, attempt quizzes, submit assignments, earn certificates, leave reviews, Q&A;

A user can be both student and instructor in the same tenant — the instructor role is additive. The role enum is extended in `internal/rbac/roles.go`; the `middleware.RequireRole` helper already accepts a variadic list, so no middleware change is needed.

9.3 Entitlement gating

Each Pro feature is gated by a single entitlement key. lastsaas's `Plan.entitlements` map is a `map[string]bool` in the `plans` collection. We seed three new plans during Phase 0 hardening:

Entitlement key	Free tier	Pro tier	Business tier
lms.core	✓	✓	✓
lms.quizzes	✓ (10 question types)	✓ (all 14)	✓ (all 14)
lms.certificates	—	✓	✓
lms.multi_instructor	—	✓	✓
lms.content_drip	—	✓	✓
lms.subscriptions	—	✓ (5 plans)	✓ (unlimited)
lms.gamification	—	✓	✓
lms.reports	—	basic	✓ advanced + export
lms.tutorai	—	100 generations/mo	1000 generations/mo

Entitlement key	Free tier	Pro tier	Business tier
lms.zoom	—	add-on	add-on
lms.notifications	in-app + email	+ push	+ push
lms.course_preview	—	✓	✓
lms.attachments	—	✓	✓
lms.lesson_notes	—	✓	✓
lms.custom_emails	—	✓	✓
lms.course_duplicate	—	✓	✓
lms.gift_course	—	✓	✓

Entitlement checks happen at two layers: the route-level middleware (coarse-grained — returns 402 if the tenant's plan lacks the entitlement) and the service-level check (fine-grained — e.g. `if !plan.lmsTutorAI { return ErrQuotaExceeded }` at the moment of AI generation).

9.4 Event catalog (new event types)

Event	Emitted by	Consumers
course.published	course service	notification, search indexer, webhook
course.unpublished	course service	search indexer, webhook
lesson.completed	enrollment service	gamification (award points), drip (unlock next), notification, certificate-issuance check
quiz.attempt.started	quiz service	notification
quiz.attempt.ended	quiz service	gamification, notification, certificate-issuance check
assignment.submitted	assignment service	notification (instructor)
assignment.evaluated	assignment service	notification (student)
enrollment.created	enrollment service	notification, gamification, webhook
enrollment.completed	enrollment service	gamification, certificate.issued (if eligible)
enrollment.expired	subscription/drip scheduler	notification
certificate.issued	certificate service	notification, webhook
coupon.redeemed	ecommerce service	coupon usage counter
order.paid	ecommerce service (Stripe webhook)	enrollment creation, revenue ledger entries
order.refunded	ecommerce service	revenue ledger reversal, enrollment cancellation

Event	Emitted by	Consumers
subscription.activated	subscription service	enrollment creation/maintenance
subscription.renewed	Stripe webhook	enrollment extension
subscription.cancelled	subscription service	enrollment expiry scheduling
subscription.payment_failed	Stripe webhook	dunning email sequence
instructor.application.submitted	instructor service	notification (admins)
instructor.application.approved	instructor service	notification (instructor), RBAC role grant
withdrawal.requested	instructor service	notification (admins)
withdrawal.paid	instructor service	notification (instructor)
drip.unlocked	drip scheduler	notification (student)
ai.generation.completed	ai service	notification (instructor)
review.posted	enrollment service	notification (instructor)
qna.question.asked	qna service	notification (instructor + enrolled students)
qna.answer.posted	qna service	notification (question author)
announcement.posted	announcement service	notification (enrolled students), email

10. Video and Media Pipeline

Video is the highest-stakes infrastructure decision in the project. Three providers are realistic candidates: **Bunny Stream** (cheapest, generous encoding, token-auth DRM), **Mux** (best DX, AI chaptering, higher per-minute cost), and **Self-hosted S3 + HLS** (most control, most ops burden). The recommendation is Bunny Stream for the SaaS launch — its token-auth signing covers 80% of the DRM-lite requirement at \$0.005 per minute streamed with no minimums, and the API is small enough that the adapter is ~200 lines of Go.

10.1 Decision matrix

Factor	Bunny Stream	Mux	Self-hosted S3+HLS
Per-minute streaming cost	\$0.005	\$0.007–\$0.014	S3 egress (~\$0.09/GB)
Encoding cost	Included	\$0.02–\$0.05/minute	Self-managed (FFmpeg)
Token-auth signed URLs	✓ native	✓ signed playback URLs	DIY
Adaptive bitrate (ABR)	✓ auto	✓ auto (best in class)	Manual renditions
Upload flow	Direct to Bunny via presigned URL	Direct upload via Mux	Presigned S3 + transcoder
Thumbnails	✓ auto-generated	✓ auto + AI chaptering	FFmpeg snippet
Watermarking	URL-based overlay	Burn-in or overlay	FFmpeg overlay
Time-to-first-frame	Good	Excellent	Depends on CDN
Lock-in risk	Low (video files are MP4 + HLS)	Medium	None
Verdict	**Recommend ed**	Better DX, 2-3x cost	Only for cost-obsessed

10.2 Upload and playback flow

The upload pipeline is a four-step dance that keeps large video bytes off the lastsaaas process entirely:

- 1 Frontend requests an upload URL.** Instructor selects a video file in the lesson editor. The frontend POSTs to `/api/v1/lms/instructor/lessons/{id}/video/upload-url` with the file size and MIME type. The backend validates the instructor owns the lesson, checks the tenant's per-tenant upload budget, and calls Bunny's API to create a video entity + generate a presigned upload URL (TTL: 1 hour).
- 2 Frontend PUTs the file directly to Bunny.** The file bytes never touch the lastsaaas process. Upload progress is reported by the browser's `XMLHttpRequest`.

-
- 3 **Frontend confirms upload.** POST to `/api/v1/lms/instructor/lessons/{id}/video/confirm` with the Bunny video ID. The backend polls Bunny until the transcode is complete (typically 30-90 seconds for a 1-hour 1080p video), fetches the thumbnail URLs, and writes the `lesson.video` subdocument.
- 4 **Student playback uses a signed URL.** When a student opens the lesson, the backend signs a Bunny playback URL with a 5-minute TTL + the student's user ID as the watermark token. The frontend `<VideoPlayer>` component renders a thin wrapper over Bunny's embed or HLS.js player with the watermark overlay (student email) drawn on a canvas.

10.3 Content security (Pro)

True DRM (Widevine, FairPlay) is out of scope for the SaaS launch — it requires certificate authorities, native apps, and per-platform SDKs. The Pro-tier content security features are the best-effort alternatives that Tutor LMS Pro itself ships:

- **Signed playback URLs** with 5-minute TTL — prevents naive link sharing.
- **Dynamic watermark** — student email rendered as a semi-transparent overlay that drifts position every 15 seconds, deterring screen-recording redistribution.
- **Domain-locked embeds** — Bunny's `allowed_referrers` configured per tenant to the tenant's custom domain.
- **Right-click + keyboard-shortcut suppression** on the player container (frontend; advisory only).
- **Device fingerprinting + concurrent-session cap** — Pro feature. Browser fingerprint stored in `device_session` collection; max N concurrent active sessions per student (default 3).
- **IP geo-fencing** — optional per-course blocklist of countries (uses Cloudflare headers).

10.4 Object storage abstraction

Beyond video, several features need general object storage: certificate PDFs, assignment submissions, course attachments, instructor signatures. We add a thin Go abstraction in `internal/storage/` with two implementations: `S3Storage` (for AWS S3 / Cloudflare R2 / MinIO via the S3 API) and `BunnyStorage` (for video assets). The interface:

```
type Storage interface { Put(ctx context.Context, key string, r io.Reader, contentType
string) (url string, err error) Get(ctx context.Context, key string) (io.ReadCloser,
error) Delete(ctx context.Context, key string) error SignedURL(ctx context.Context, key
string, ttl time.Duration) (string, error) }
```


11. Phased 6-Month Roadmap

Five phases, twenty-four weeks. Each phase has explicit exit criteria — no phase exits until its criteria are demonstrably met in a staging demo. Weeks are calendar weeks, not working weeks; assume a four-day working week per engineer. Team composition assumed: two backend engineers, two frontend engineers, one designer (part-time), one tech lead (you). Phase 0 work overlaps with hiring and onboarding.

11.1 Phase overview

Phase	Weeks	Theme	Headline deliverable
0	0 (parallel)	Foundation hardening	Object storage, scheduled jobs, RBAC roles, entitlement seeds, migration runner
1	1–6	Core LMS	Course catalog + curriculum builder + lesson player + student enrollment + progress tracking — demoable course sale + take
2	7–12	Ecommerce	Cart + checkout + Stripe + coupons + invoices + instructor earnings + withdrawal flow
3	13–16	Pro authoring	Assignments + certificates (12 templates) + content drip + multi-instructor + revenue share
4	17–20	Pro engagement	Gamification + Q&A; + announcements + lesson notes + notifications (push) + course preview + course attachments
5	21–24	Reports + AI + launch	Sales/enrollment/completion reports + instructor analytics + TutorAI course/quiz generation + load test + launch hardening

11.2 Phase 0 — Foundation hardening (Week 0, parallel)

Phase 0 is the only phase that modifies lastsaas's existing primitives. It runs in parallel with hiring and onboarding. By the end of Week 0, the team can write a new LMS module that uses object storage, scheduled jobs, the new RBAC roles, and the new entitlement keys without modifying any lastsaas core file.

Scope

- Add `internal/storage/` with S3 + Bunny implementations; inject via dependency injection in `cmd/server/main.go`.
- Add `internal/jobs/` — a river-style job runner backed by MongoDB. Collections: `jobs` (pending + `retry_count` + `run_after`), `job_runs` (history). One worker goroutine per job type; leases prevent double-execution. Job types: `drip.unlock_check`, `subscription.dunning`, `certificate.reissue`, `ai.generate_course`, `report.aggregate_daily`.
- Add `instructor` and `student` to RBAC role enum. Update `tenant_memberships` to support role list. Update `middleware.RequireRole` to accept both.
- Seed new entitlement keys into the three new plan documents (Free / Pro / Business) in `plans` collection.
- Add `internal/migrations/` — a tiny migration runner. Each migration is a Go file implementing `Up(ctx, db);` tracked by a `_migrations` collection.

- Add MongoDB schema validators for all new collections (Phase 1 onward).
- Add `internal/lms/shared/` package with shared types, error codes, and helpers.

Exit criteria

- An engineer can run `make dev` and exercise a stub LMS endpoint that uses storage, jobs, and entitlement middleware.
- An end-to-end test creates a tenant, signs up a student, promotes them to instructor, and verifies role checks.
- Migration runner successfully applies a no-op migration and records it in `_migrations`.

11.3 Phase 1 — Core LMS (Weeks 1–6)

By the end of Phase 1, a student can browse a catalog, view a course detail page, enroll for free or via a stub checkout, watch a video lesson, mark it complete, and see their progress. An instructor can create a course with topics, lessons, and quizzes; publish it; and see enrollments. This is the minimum that proves the architecture works end-to-end.

Week-by-week

Week	Backend deliverables	Frontend deliverables
1	Course + Topic + Lesson models, CRUD endpoints, category/tag taxonomies, slug generation	CourseCard component, CoursesPage (catalog), basic CourseDetailPage
2	Curriculum tree endpoints (GET/PUT), drag-drop reorder support, lesson type detection	CurriculumBuilder component (dnd-kit), LessonTypeSwitcher
3	Enrollment model, create/list/progress endpoints, lesson_progress tracking	MyCoursesPage, LearningPage skeleton (sidebar + content area)
4	Video upload-url + confirm endpoints (Bunny adapter), signed playback URL endpoint	component (Bunny embed + watermark overlay), VideoUploader
5	Quiz model, attempts, answer submission, scoring for auto-graded question types (10 free types)	QuizRenderer with per-type renderers (8 free types — fill_in_blank + matching deferred), QuizTimer
6	Course review + rating aggregation, Q&A; foundation, polish, bug bash	Reviews tab, Ask Question form, integration tests pass

Exit criteria

- End-to-end test: instructor creates a course with 2 topics, 5 lessons (3 video, 1 text, 1 quiz with 5 single-choice questions), publishes, student enrolls, completes all lessons, passes the quiz, sees 100% progress.
- Catalog page loads in < 200ms p95 with 100 published courses.
- Lesson video plays with watermark overlay; right-click is suppressed; signed URL expires correctly.

11.4 Phase 2 — Ecommerce (Weeks 7–12)

Phase 2 turns the LMS into a business. By the end of the phase, a student can add courses to a cart, apply a coupon, check out via Stripe (one-time or recurring subscription plan), receive an invoice, and the instructor sees their share of revenue in a ledger. The platform operator can issue refunds; revenue reversals flow back through

the ledger.

Week-by-week

Week	Backend deliverables	Frontend deliverables
7	Cart model + endpoints, coupon model + validation, order model + order_items + order_activities	CartDrawer, CartPage, CouponInput, CheckoutForm (billing fields)
8	Stripe Checkout integration (reuse stripe.Service), line-item shape for course / bundle / plan, success/cancel URL routing	CheckoutPage end-to-end, OrderReceipt component, redirect handling
9	Stripe webhook handler (idempotent), order.paid event, enrollment creation on payment, revenue_ledger entries	MyOrdersPage, invoice download (existing gofpdf)
10	Subscription plan CRUD, subscription creation on recurring checkout, subscription.activated event, enrollment extension	PlanCard, PlanPicker, MySubscriptionsPage, cancel/resume flows
11	Instructor earnings report (own courses), revenue share computation on order.paid, withdrawal request + admin approval flow	EarningsPage with chart, WithdrawalsPage, admin OrdersPage + refund dialog
12	Coupon redemption counters, expired coupon handling, tax integration (Stripe Tax recommended), polish, bug bash	Admin CouponsPage, coupon usage stats, end-to-end checkout tests with Stripe test mode

Exit criteria

- End-to-end Stripe test-mode checkout: add course → apply coupon → pay → enrollment created → revenue ledger shows instructor share → refund → ledger reversed → enrollment cancelled.
- Subscription lifecycle: subscribe → renew (Stripe test trigger) → cancel → dunning email → expire.
- Instructor can request withdrawal; admin can approve; ledger shows payout as a debit.
- Invoice PDF download works with correct tax line items.

11.5 Phase 3 — Pro authoring (Weeks 13–16)

Phase 3 unlocks the Pro entitlement gates. This is the phase that converts free-tier tenants to paid tiers.

Week-by-week

Week	Backend deliverables	Frontend deliverables
13	Assignment model, submission upload (storage abstraction), instructor grading, assignment.evaluated event	AssignmentUploader (drag-drop + progress), AssignmentGradeCard
14	Certificate template model (HTML+CSS), Playwright-based PDF rendering service, certificate issuance on enrollment.completed, verification_hash, public verification endpoint	CertificatePreview (live HTML render), CertificateVerifyPage (public)
15	Drip rule model, drip.unlock_check job, drip.unlocked event, /student/lessons/{id}/drip-status endpoint	DripRuleEditor (3 modes: schedule / prerequisite / enrollment-days), DripBadge on locked lessons

Week	Backend deliverables	Frontend deliverables
16	Instructor application flow, admin approval, course_instructor N:N, revenue_share_percent, Course_Duplicator, polish	InstructorPicker, RevenueShareEditor, ApplyToBeInstructorPage, admin InstructorApplicationsPage

Exit criteria

- Course with 3 instructors at 50/30/20 split; checkout generates correct revenue ledger entries; partial refund reverses proportionally.
- Certificate auto-issues on course completion; PDF renders correctly with template 1 (default) and template 5 (customizable); verification URL works unauthenticated.
- Content drip unlocks lessons on schedule, on prerequisite completion, and on enrollment-day count.
- Assignment submission + grading flow works end-to-end with file upload.

11.6 Phase 4 — Pro engagement (Weeks 17–20)

Phase 4 adds the engagement layer that makes courses sticky: gamification, social Q&A, notifications, lesson notes, course preview, and downloadable attachments. Most of this phase is frontend-heavy; the backend is largely event subscriptions off the catalog built in Phases 1–3.

Week-by-week

Week	Backend deliverables	Frontend deliverables
17	Gamification points engine (event→points rules), badge criteria evaluation job, leaderboard rebuild job	PointsDisplay, BadgeWall, LeaderboardTable (course + tenant scope)
18	Q&A; endpoints (already roughed in Phase 1) fleshed out with answers, upvotes, instructor-answered badge; announcements model + email-out	QnaThreadedView, AnnouncementList, instructor AnnouncementComposer
19	Lesson notes model + endpoints; notifications model + dispatcher (email + in-app + Pusher for push); notification_preference per user	LessonNotesPanel (sidebar in player), NotificationBell with realtime feed, PreferenceMatrix
20	Course preview (mark lessons as preview, allow unauthenticated watch); course attachments (downloadable resources, gated by enrollment); polish	PreviewBadge, AttachmentList, download link with signed URL

Exit criteria

- Student earns 100 points for course completion, sees badge awarded in real-time via Pusher.
- Notification bell shows unread count; clicking takes student to the relevant lesson / certificate / announcement.
- Lesson notes save and load correctly; survives lesson navigation.
- Preview lessons play without auth; non-preview lessons show login CTA.

11.7 Phase 5 — Reports + AI + launch (Weeks 21–24)

Phase 5 closes the gap to Tutor LMS Pro parity with reports and AI course generation, then enters launch hardening. The AI work depends on a separate LLM provider decision (see Chapter 12) and is the most likely to slip — it's scoped to be cuttable without blocking launch.

Week-by-week

Week	Backend deliverables	Frontend deliverables
21	Report aggregation jobs (daily rollups), admin reports endpoints (overview, sales, enrollments, completion), instructor reports (own earnings, students)	ReportsPage with KPI cards + Recharts line/bar/pie; export to CSV
22	TutorAI: generate-course-outline (LLM call with structured output), generate-lesson (per outline item), generate-quiz (per lesson)	AiStudioPanel with prompt input + preview + insert-into-builder; JobProgress component
23	Performance: index audit, query optimization, N+1 elimination; load test with k6 (1000 concurrent students, 50 instructors)	Performance fixes; bundle-size audit; code-split Instructor + Admin routes
24	Security review (RBAC matrix audit, entitlement bypass check, SQL/NoSQL injection sweep, secrets scan); launch runbook; on-call setup	Polish, accessibility audit (axe-core), browser matrix testing, final bug bash

Exit criteria

- Reports page loads in < 500ms with 10k orders + 5k enrollments; CSV export works.
- TutorAI generates a usable course outline from a 1-sentence prompt in < 30 seconds; instructor can edit + insert into builder.
- k6 load test passes: 1000 concurrent students browsing catalog, 100 concurrent video playbacks, 10 concurrent checkouts — p95 latency < 1s.
- No critical security findings; launch runbook documented; first customer onboarded.

12. Risk Register and Mitigations

#	Risk	Pr ob	Im pa ct	Mitigation	Owner
R 1	Video streaming cost overrun (Bunny bill spikes from a viral course)	Med	High	Per-tenant bandwidth budget (configstore); alert at 80% of cap; hard cap at 120% with degraded playback quality	Tech lead
R 2	Stripe subscription dunning failures (SCA, expired cards) silently churn subscribers	High	High	Stripe Smart Retries + our own 3-attempt email sequence; subscription.payment_failed event triggers dunning; daily report of at-risk subs	Backend
R 3	MongoDB schema drift across environments (no migration framework historically)	High	Med	Phase 0 introduces migration runner; every PR that touches a collection must add a migration; CI fails if migrations are pending	Backend
R 4	Certificate PDF rendering inconsistency across Playwright versions	Med	Med	Pin Playwright version in Dockerfile; render certificates server-side only; cache rendered PDF in object storage; re-render only on template change	Backend
R 5	Multi-instructor revenue math breaks under partial refunds + bundle purchases	Med	High	Model revenue as double-entry ledger from day one; property tests cover: partial refund, bundle refund, refund after instructor payout (clawback)	Backend
R 6	Content drip scheduler fires at wrong time across student timezones	Med	Med	Store all drip times as UTC; student sees localized time on frontend; scheduler runs in UTC; document the model clearly in API docs	Backend
R 7	TutorAI generation quality varies wildly across LLM providers and prompt phrasing	High	Med	Ship as Pro add-on with explicit 'draft' framing; allow instructor to edit everything; track acceptance rate; iterate prompts monthly	Tech lead
R 8	RBAC permission leakage (instructor sees another instructor's course)	Med	High	Every repository method takes actor_id and filters by ownership; integration tests assert cross-tenant + cross-instructor isolation; quarterly security audit	Backend
R 9	Catalog search latency degrades as courses grow past 10k	Med	Low	Phase 1: MongoDB text index covers 10k; Phase 5: revisit with Atlas Search or Meilisearch if p95 > 500ms	Backend
R 10	Stripe webhook duplicates cause double-enrollment or double ledger entries	Low	High	Webhook handlers idempotent on Stripe event ID; idempotency key stored in webhook_events collection; replay tests in CI	Backend
R 11	Object storage cost grows unbounded (years of video + certificate PDFs)	Med	Med	Lifecycle policies: move cancelled-tenant assets to cold storage after 90 days; delete after 365 days; tenant export + delete on churn	Tech lead

#	Risk	Pr ob	Im pa ct	Mitigation	Owner
R 1 2	Instructor payout compliance (tax forms, KYC, regional payout methods)	Hig h	Hig h	Defer to Stripe Connect Express for Phase 1–3 (handles KYC, 1099s, payout scheduling); only build custom payout if Stripe Connect is unavailable in a target market	Tech lead

13. Open Questions and Next Steps

13.1 Decisions to confirm before Phase 1 starts

#	Decision	Options	Recommendation	Decision needed by
D 1	Video provider	Bunny Stream / Mux / S3+HLS	Bunny Stream (lowest cost, token-auth, sufficient DRM-lite)	End of Phase 0
D 2	AI provider for TutorAI	OpenAI / Anthropic / Gemini / z-ai-web-dev-sdk	z-ai-web-dev-sdk if available — lowest integration cost; else OpenAI for streaming + tool use	End of Phase 4
D 3	i18n strategy	None / Lingui / i18next / next-intl	Defer — Phase 5+ if early customers need non-English schools	Post-launch
D 4	Search backend	Mongo text index / Atlas Search / Meilisearch	Mongo text index for MVP; revisit at 10k courses	Phase 5 review
D 5	Marketplace mode (Udemy-style cross-tenant catalog)	Defer / Build now	Defer — current scope is per-tenant LMS, not marketplace	Post-launch
D 6	Mobile app	No / React Native / PWA only	PWA only — React 19 + Vite already produces an installable PWA; native app deferred	Post-launch
D 7	Email provider	Resend (existing) / Postmark / SES	Keep Resend — already integrated, sufficient for LMS volume	Confirmed
D 8	Realtime push (notifications, live Q&A;)	Pusher / self-hosted Centrifugo / WebSocket-only	Pusher for Phase 4 (fastest TTM); Centrifugo if cost becomes an issue	End of Phase 3

13.2 Team and resourcing

The roadmap assumes a team of five: two backend engineers, two frontend engineers, one designer (part-time), one tech lead (you, full-time). The team can be smaller if Phase 0 ships before hiring — Phase 0 is tech-lead-only work. The team can be larger if you want to compress the timeline: a third backend engineer in Phase 2 (parallelizing ecommerce and subscription work) and a third frontend engineer in Phase 4 (parallelizing gamification, notifications, and course preview) would compress the 24-week plan to ~18 weeks.

Critical hire: a backend engineer with Stripe Billing experience. Subscription dunning, proration, and refund cascades are the highest-risk backend work in the project; an engineer who has built this before will save 2–3 weeks in Phase 2.

13.3 Immediate next steps (this week)

- 1 Confirm the stack additions (Bunny Stream, Pusher, Playwright for certificates, z-ai-web-dev-sdk for AI) and provision accounts.
- 2 Stand up a staging environment on Fly.io with a MongoDB Atlas M10 cluster and a Bunny Stream library.

-
- 3 Begin Phase 0 work in a feature branch off main; create the `internal/storage/`, `internal/jobs/`, and `internal/migrations/` packages.
 - 4 Open the codewiki MCP for lastsaaas (<https://codewiki.google/github.com/jonradoff/lastsaas>) and use it as the onboarding doc for new engineers — it walks them through the codebase faster than reading source.
 - 5 Post the engineering plan (this document) to the team channel; collect feedback for one week; freeze scope at end of week.
 - 6 Begin hiring (if not already started) — prioritize the Stripe-experienced backend engineer first.

13.4 What this plan deliberately does NOT include

To keep the 24-week timeline realistic, the following are explicitly out of scope for v1 launch and should be revisited post-launch:

- Native mobile apps (PWA only).
- Marketplace mode (cross-tenant catalog, transaction fees).
- SCORM / LTI / xAPI compliance (deferred until a customer asks).
- Full DRM (Widevine / FairPlay) — Bunny token-auth is the v1 ceiling.
- Real-time co-authoring of courses (Tutor's multi-instructor is sequential edits only).
- Built-in live class tool (we integrate Zoom; we do not build our own).
- AI-powered personalized learning paths (TutorAI generates content; adaptive sequencing is a v2).
- Built-in discussion forums (Q&A; is per-lesson; full forums are deferred — BuddyPress parity is a v2 if customers ask).
- WCAG 2.1 AA compliance audit (we will follow accessibility best practices; formal audit deferred).
- Multi-currency settlement (display in any currency; settled in USD only at v1).